Zonas de Ribera en Burgos: Ríos Arlanzón, Vena y Ubierna

Flujo de trabajo GIS en R — Shapefile IGN/CNIG → Buffer 60 m → KMZ

Antonio Canepa

2026-03-12

Tabla de contenidos

1. Introducción **2**
1.1. Fuentes de datos 2

2. Librerías necesarias **2**

3. Configuración de rutas **2**

4. Exploración previa del shapefile **3**

5. Identificar la columna de nombres **4**

6. Lectura del shapefile **4**

7. Filtrado de los tres ríos **5**

8. Limpieza de columnas **5**

9. Reproyección a UTM zona 30N **5**

10. Homogeneización de geometrías **7**

11. Buffer de 60 metros **7**

12. Disolución de polígonos por río **7**

13. Reproyección a WGS84 **8**

14. Metadatos y estilos de color **8**

15. Verificación final y visualización **8**

16. Exportación a KML **10**

17. Compresión a KMZ **10**

18. Resumen del flujo de trabajo **11**

19. Información de sesión **11**

1. Introducción

Este documento describe paso a paso un flujo de trabajo completo en **R** para:

1. Leer un **shapefile** de la red hidrográfica descargado del IGN/CNIG.
2. Filtrar los cauces del **Río Arlanzón**, **Río Vena** y **Río Ubierna** dentro de la provincia de Burgos.
3. Aplicar un **buffer de 60 metros** para delimitar la zona de ribera.
4. Disolver y reproyectar la capa resultante.
5. Exportar el resultado final a formato **KMZ** (Google Earth / QGIS).

   Nota: El parámetro `eval: false` en el encabezado YAML desactiva la ejecución automática del código al renderizar. Cámbialo a `eval: true` una vez que el shapefile esté disponible en las rutas indicadas.

1.1. Fuentes de datos

Los shapefiles de hidrografía pueden descargarse desde:

- MíRAme CHD — Confederación Hidrográfica del Duero
- Centro de Descargas del CNIG (IGN)
- MITECO — Red Hidrográfica

---

2. Librerías necesarias

Se utilizan tres paquetes fundamentales:

Paquete	Función principal en este flujo
`sf`	Lectura, reproyección, buffer y exportación de datos vectoriales (OGC)
`tidyverse`	Filtrado, selección y mutación de tablas de atributos con verbos dplyr
`zip`	Compresión del archivo KML generado en un contenedor KMZ

```r
library(sf)        # Lectura, manipulación y exportación de datos espaciales
library(tidyverse) # Filtrado y manipulación de tablas de atributos
library(zip)       # Compresión del KML en KMZ
```

---

3. Configuración de rutas

Define la **ruta local** al shapefile descomprimido. El nombre del archivo varía según el producto descargado:

Producto IGN	Nombre típico del .shp
IGR Hidrografía INSPIRE	`HY_PhysicalWaters_Watercourses.shp`
BTN25	`HID_Hidrografia_Cauce.shp`
BCN200	`hidrografia_l.shp`
CHD (Duero)	`Duero_Rios_Duero_YYYYMMDD.shp`

```r
# CAMBIA esta ruta por la ubicación real en tu equipo:
ruta_shp <- "Duero_Rios_Duero_20260312/Duero_Rios_Duero_20260312.shp"
```

```r
# Alternativa BTN25:
# ruta_shp <- "C:/datos/btn25/HID_Hidrografia_Cauce.shp"
```

---

4. Exploración previa del shapefile

Antes de filtrar, inspeccionamos el shapefile para identificar:

- El **nombre exacto** de la columna que contiene los nombres de los ríos.
- El **formato tipográfico** de los nombres (tildes, mayúsculas, prefijos, etc.).

La clave es el argumento `query =` de `st_read()`, que acepta SQL estándar (`SELECT ... LIMIT 20`) y evita cargar el archivo completo en memoria — muy útil cuando el shapefile pesa varios cientos de megabytes.

```r
# Leemos solo las primeras 20 filas usando SQL
cabecera <- st_read(
  ruta_shp,
  query = 'SELECT * FROM "Duero_Rios_Duero_20260312" LIMIT 20'
)
```

```
#> Reading query `SELECT * FROM "Duero_Rios_Duero_20260312" LIMIT 20'
#> from data source `/home/antonio/Documents/Dropbox/R - SUPPORT/CITIZEN-SCIENCE-DATA/Rios_Burgos_R/Duer
#>   using driver `ESRI Shapefile'
#> Simple feature collection with 20 features and 3 fields
#> Geometry type: LINESTRING
#> Dimension:     XY
#> Bounding box:  xmin: 263632.6 ymin: 4544650 xmax: 464479.1 ymax: 4745133
#> Projected CRS: ETRS89 / UTM zone 30N
```

```r
# Columnas disponibles
cat("Columnas disponibles en el Shapefile:\n")
```

```
#> Columnas disponibles en el Shapefile:
```

```r
print(names(cabecera))
```

```
#> [1] "id"             "nombre"         "length"         "_ogr_geometry_"
```

```r
# Muestra de nombres de ríos
cat("\nMuestra de nombres de ríos:\n")
```

```
#>
#> Muestra de nombres de ríos:
```

```r
print(head(cabecera$nombre, 20))
```

```
#>  [1] "Arroyo De La Huerga"
#>  [2] "Colector de la Gavia o de la Nava Nestosa"
#>  [3] "Río Odra"
#>  [4] "Arroyo De Alcedo"
#>  [5] "Sn"
#>  [6] "Sn"
#>  [7] "Arroyo De Riofresno"
#>  [8] "Río Zapardiel (brazo)"
#>  [9] "Río Zapardiel (brazo)"
#> [10] "Río Zapardiel (brazo)"
#> [11] "Río Zapardiel (brazo)"
#> [12] "Río Zapardiel (brazo)"
#> [13] "Río Zapardiel (brazo)"
```

```
#> [14] "Arroyo De La Majada"
#> [15] "Sn"
#> [16] "Arroyo De La Granja O Del Valle"
#> [17] "Río Seco"
#> [18] "Sn"
#> [19] "Sn"
#> [20] "Arroyo Del Henar"
```

---

5. Identificar la columna de nombres

Según el producto descargado, la columna con el nombre del río puede llamarse de forma diferente. Asignamos su nombre a la variable `col_nombre` para usarla dinámicamente en los pasos posteriores con `.data[[col_nombre]]`.

Producto	Nombre de columna habitual
IGR Hidrografía	`localname` o `nombre`
BTN25	`NOMBRE`
MITECO	`NOM_RIO`
OpenStreetMap	`name`

```r
col_nombre <- "nombre"  # <-- AJUSTA según tu shapefile
```

---

6. Lectura del shapefile

Leemos el shapefile completo con `st_read()`. En el código original, el filtro espacial por bounding box de Burgos está comentado para mayor flexibilidad; puede reactivarse (líneas con `wkt_filter`) para ahorrar memoria cuando el dataset cubre toda España.

El filtro WKT define un rectángulo que envuelve la provincia de Burgos:

```
POLYGON((-4.10 41.80, -2.90 41.80, -2.90 43.00, -4.10 43.00, -4.10 41.80))
```

```r
# Bounding box de Burgos como WKT (opcional, descomenta para activar)
# bbox_burgos_wkt <- "POLYGON((-4.10 41.80, -2.90 41.80, -2.90 43.00,
#                              -4.10 43.00, -4.10 41.80))"

message(">>> Leyendo shapefile...")
Rios_Burgos <- hidrografia_burgos <- st_read(ruta_shp)
```

```
#> Reading layer `Duero_Rios_Duero_20260312' from data source
#>   `/home/antonio/Documents/Dropbox/R - SUPPORT/CITIZEN-SCIENCE-DATA/Rios_Burgos_R/Duero_Rios_Duero_20
#>   using driver `ESRI Shapefile'
#> Simple feature collection with 39683 features and 3 fields
#> Geometry type: MULTILINESTRING
#> Dimension:     XY
#> Bounding box:  xmin: 109691.5 ymin: 4452427 xmax: 599820.6 ymax: 4776178
#> Projected CRS: ETRS89 / UTM zone 30N
```

```r
# Con filtro espacial:
# Rios_Burgos <- st_read(ruta_shp, wkt_filter = bbox_burgos_wkt)

message(">>> Elementos cargados: ", nrow(Rios_Burgos))
```

```
# cat("\nRíos disponibles en la zona de Burgos:\n")
# print(sort(unique(Rios_Burgos[[col_nombre]])))
```

---

7. Filtrado de los tres ríos

Usamos `grepl()` con `ignore.case = TRUE` para una búsqueda robusta que no depende de mayúsculas ni tildes exactas. El operador `|` dentro de `filter()` selecciona cualquier fila que coincida con al menos uno de los tres patrones.

```
rios_interes <-
  Rios_Burgos %>%
  filter(
    grepl("Río Arlanzón", .data[[col_nombre]], ignore.case = TRUE) |
    grepl("Río Vena",     .data[[col_nombre]], ignore.case = TRUE) |
    grepl("Río Ubierna",  .data[[col_nombre]], ignore.case = TRUE)
  )

message(">>> Segmentos encontrados: ", nrow(rios_interes))

cat("\nRíos filtrados:\n")
```

```
#>
#> Ríos filtrados:
```

```
print(table(rios_interes[[col_nombre]]))
```

```
#>
#>         Río Arlanzón Río Arlanzón (Brazo)          Río Ubierna
#>                    1                    11                    1
#>             Río Vena
#>                    1
```

---

8. Limpieza de columnas

Conservamos únicamente las columnas imprescindibles (`nombre` y `geometry`) para reducir el peso del objeto en memoria y simplificar los pasos posteriores.

```
rios_limpios <- rios_interes %>%
  select(nombre, geometry)
```

---

9. Reproyección a UTM zona 30N

Los buffers en metros exigen un **sistema de coordenadas proyectado**. El estándar oficial para España peninsular es **EPSG:25830** (ETRS89 / UTM zona 30N). Proyectar sobre datos geográficos (grados) generaría distancias incorrectas.

```
cat("\nCRS original del shapefile:\n")
```

```
#>
#> CRS original del shapefile:
```

```r
print(st_crs(rios_limpios))
```

```
#> Coordinate Reference System:
#>   User input: ETRS89 / UTM zone 30N
#>   wkt:
#> PROJCRS["ETRS89 / UTM zone 30N",
#>     BASEGEOGCRS["ETRS89",
#>         ENSEMBLE["European Terrestrial Reference System 1989 ensemble",
#>             MEMBER["European Terrestrial Reference Frame 1989"],
#>             MEMBER["European Terrestrial Reference Frame 1990"],
#>             MEMBER["European Terrestrial Reference Frame 1991"],
#>             MEMBER["European Terrestrial Reference Frame 1992"],
#>             MEMBER["European Terrestrial Reference Frame 1993"],
#>             MEMBER["European Terrestrial Reference Frame 1994"],
#>             MEMBER["European Terrestrial Reference Frame 1996"],
#>             MEMBER["European Terrestrial Reference Frame 1997"],
#>             MEMBER["European Terrestrial Reference Frame 2000"],
#>             MEMBER["European Terrestrial Reference Frame 2005"],
#>             MEMBER["European Terrestrial Reference Frame 2014"],
#>             ELLIPSOID["GRS 1980",6378137,298.257222101,
#>                 LENGTHUNIT["metre",1]],
#>             ENSEMBLEACCURACY[0.1]],
#>         PRIMEM["Greenwich",0,
#>             ANGLEUNIT["degree",0.0174532925199433]],
#>         ID["EPSG",4258]],
#>     CONVERSION["UTM zone 30N",
#>         METHOD["Transverse Mercator",
#>             ID["EPSG",9807]],
#>         PARAMETER["Latitude of natural origin",0,
#>             ANGLEUNIT["degree",0.0174532925199433],
#>             ID["EPSG",8801]],
#>         PARAMETER["Longitude of natural origin",-3,
#>             ANGLEUNIT["degree",0.0174532925199433],
#>             ID["EPSG",8802]],
#>         PARAMETER["Scale factor at natural origin",0.9996,
#>             SCALEUNIT["unity",1],
#>             ID["EPSG",8805]],
#>         PARAMETER["False easting",500000,
#>             LENGTHUNIT["metre",1],
#>             ID["EPSG",8806]],
#>         PARAMETER["False northing",0,
#>             LENGTHUNIT["metre",1],
#>             ID["EPSG",8807]]],
#>     CS[Cartesian,2],
#>         AXIS["(E)",east,
#>             ORDER[1],
#>             LENGTHUNIT["metre",1]],
#>         AXIS["(N)",north,
#>             ORDER[2],
#>             LENGTHUNIT["metre",1]],
#>     USAGE[
#>         SCOPE["Engineering survey, topographic mapping."],
#>         AREA["Europe between 6°W and 0°W: Faroe Islands offshore; Ireland - offshore; Jan Mayen - off
#>         BBOX[35.26,-6,80.49,0.01]],
#>     ID["EPSG",25830]]
```

```r
rios_utm <- st_transform(rios_limpios, crs = 25830)
message(">>> Reproyectado a EPSG:25830 (ETRS89 / UTM zona 30N)")
```

---

10. Homogeneización de geometrías

Un shapefile de cauces puede mezclar geometrías `LINESTRING` (un segmento) y `MULTILINESTRING` (varios segmentos agrupados). `st_buffer()` y `st_union()` requieren que todas las geometrías sean del mismo tipo; `st_cast()` normaliza todo a `MULTILINESTRING` sin pérdida de información.

```r
rios_utm <- st_cast(rios_utm, "MULTILINESTRING")
message(">>> Geometrías homogeneizadas a MULTILINESTRING")
```

---

11. Buffer de 60 metros

`st_buffer()` genera un polígono que envuelve cada línea del cauce a la distancia indicada. Los parámetros `endCapStyle = "ROUND"` y `joinStyle = "ROUND"` producen geometrías suaves, más adecuadas para representar zonas de ribera naturales que los extremos planos (`FLAT`) o las esquinas angulosas (`MITRE`).

```r
rios_buffer_utm <- st_buffer(
  rios_utm,
  dist          = 60,      # 60 metros = zona de ribera estimada
  endCapStyle   = "ROUND", # Extremos redondeados
  joinStyle     = "ROUND"  # Uniones suaves entre segmentos
)

message(">>> Buffer de 60 m aplicado")
```

---

12. Disolución de polígonos por río

Cuando un cauce está representado por múltiples segmentos, el buffer produce polígonos parcialmente solapados. `st_union()` dentro de `summarise()` fusiona todos los polígonos del mismo río en un único `MULTIPOLYGON`, eliminando solapamientos internos y generando una zona de ribera continua.

```r
rios_disueltos <-
  rios_buffer_utm %>%
  group_by(nombre) %>%
  summarise(
    geometry = st_union(geometry),
    .groups  = "drop"
  )

message(">>> Polígonos disueltos: ", nrow(rios_disueltos), " entidades")
print(rios_disueltos)
```

```
#> Simple feature collection with 4 features and 1 field
#> Geometry type: GEOMETRY
#> Dimension:     XY
#> Bounding box:  xmin: 403614.1 ymin: 4662353 xmax: 482318.8 ymax: 4720276
#> Projected CRS: ETRS89 / UTM zone 30N
#> # A tibble: 4 x 2
```

```
#>   nombre                                                          geometry
#>   <chr>                                                     <GEOMETRY [m]>
#> 1 Río Arlanzón         POLYGON ((403842.2 4662555, 403841.3 4662556, 403839.3 4~
#> 2 Río Arlanzón (Brazo) MULTIPOLYGON (((411913.9 4668087, 411901 4668058, 411895~
#> 3 Río Ubierna          POLYGON ((440483.2 4693788, 440508.1 4693804, 440528 469~
#> 4 Río Vena             POLYGON ((446895.4 4690484, 446893 4690483, 446886.4 469~
```

---

13. Reproyección a WGS84

El formato KML/KMZ exige coordenadas en **EPSG:4326** (WGS84, grados decimales). Esta reproyección es obligatoria; sin ella, Google Earth o QGIS mostrarán la capa en una posición incorrecta.

```r
rios_wgs84 <- st_transform(rios_disueltos, crs = 4326)
message(">>> Reproyectado a WGS84 para exportación KML")
```

---

14. Metadatos y estilos de color

Añadimos dos columnas auxiliares:

- **`descripcion`:** texto informativo que Google Earth muestra al hacer clic sobre cada polígono.
- **`OGR_STYLE`:** cadena de estilo que el driver KML de GDAL/OGR interpreta para colorear cada río de forma diferente (azul → Arlanzón, verde → Vena, naranja → Ubierna). El prefijo hexadecimal de 2 dígitos en `fc:` controla la opacidad del relleno (`66` 40 %).

```r
rios_wgs84 <-
  rios_wgs84 %>%
  mutate(
    descripcion = paste0(
      "Zona de ribera del río ", nombre,
      " (buffer 60 m) | Fuente: IGN/CNIG"
    ),
    OGR_STYLE = case_when(
      nombre == "Río Arlanzón"        ~ "BRUSH(fc:#661E90FF);PEN(c:#1E90FF,w:2px)",
      nombre == "Río Arlanzón (Brazo)"~ "BRUSH(fc:#661E90FF);PEN(c:#1E90FF,w:2px)",
      nombre == "Río Vena"            ~ "BRUSH(fc:#6632CD32);PEN(c:#32CD32,w:2px)",
      nombre == "Río Ubierna"         ~ "BRUSH(fc:#66FF8C00);PEN(c:#FF8C00,w:2px)",
      TRUE                            ~ "BRUSH(fc:#660000FF);PEN(c:#0000FF,w:2px)"
    )
  )
```

---

15. Verificación final y visualización

Antes de exportar, comprobamos la coherencia del objeto: número de entidades, nombres incluidos, CRS y extensión espacial. La visualización con `ggplot2` + `geom_sf()` permite confirmar visualmente que los buffers tienen la forma y la posición esperadas.

```r
cat("\n===== RESUMEN FINAL =====\n")
```

```
#>
#> ===== RESUMEN FINAL =====
```

```r
cat("Número de ríos:", nrow(rios_wgs84), "\n")
```

```
#> Número de ríos: 4
```

```r
cat("Ríos incluidos:", paste(rios_wgs84$nombre, collapse = ", "), "\n")
```

```
#> Ríos incluidos: Río Arlanzón, Río Arlanzón (Brazo), Río Ubierna, Río Vena
```

```r
cat("CRS:", st_crs(rios_wgs84)$input, "\n")
```

```
#> CRS: EPSG:4326
```

```r
cat("Bounding box:\n")
```

```
#> Bounding box:
```

```r
print(st_bbox(rios_wgs84))
```

```
#>      xmin      ymin      xmax      ymax
#> -4.165801 42.107363 -3.214151 42.632870
```

```r
col_relleno <- c(
  "Río Arlanzón"        = "#1E90FF",
  "Río Arlanzón (Brazo)"= "#1E90FF",
  "Río Vena"            = "#32CD32",
  "Río Ubierna"         = "#FF8C00"
)

ggplot() +
  geom_sf(
    data     = rios_wgs84,
    aes(fill = nombre, color = nombre),
    linewidth = 0.4
  ) +
  scale_fill_manual(values  = col_relleno, na.value = "#0000FF") +
  scale_color_manual(values = col_relleno, na.value = "#0000FF") +
  coord_sf(expand = FALSE) +
  theme_minimal() +
  labs(
    title    = "Zonas de ribera (buffer 60 m)",
    subtitle = "Río Arlanzón, brazo, Vena y Ubierna",
    fill     = "Río",
    color    = "Río"
  )
```

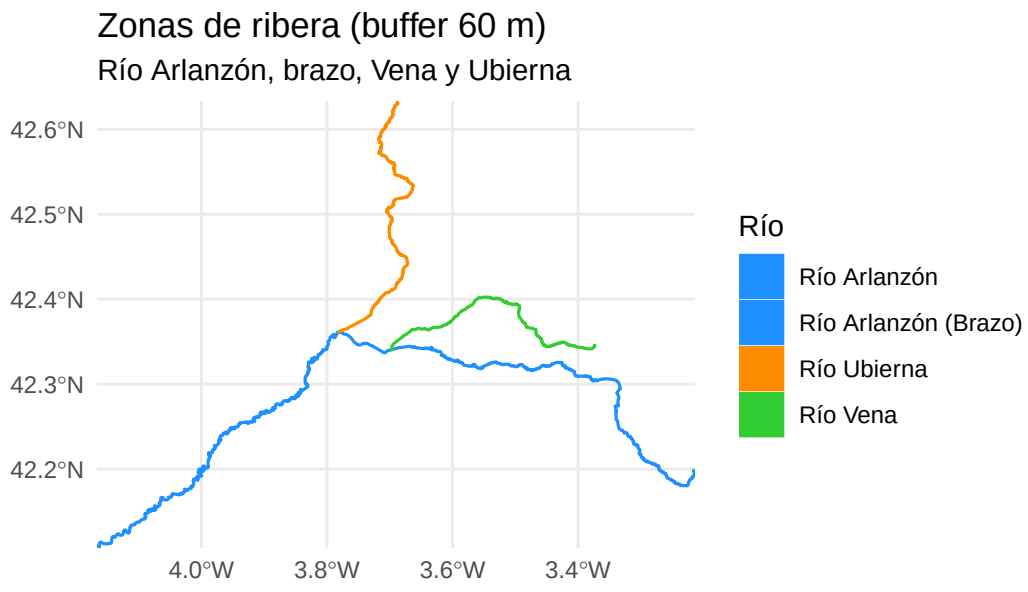

Figura 1: Zonas de ribera (buffer 60 m) de los ríos Arlanzón, Vena y Ubierna

---

16. Exportación a KML

`st_write()` llama internamente al driver KML de GDAL. El argumento `delete_dsn = TRUE` sobreescribe el archivo si ya existe, evitando el error *"Layer already exists"* en ejecuciones repetidas. La capa `OGR_STYLE` creada en el paso anterior es leída automáticamente por el driver y aplica los colores en el KML resultante.

```
ruta_kml <- "rios_burgos_buffer.kml"

st_write(
  obj        = rios_wgs84,
  dsn        = ruta_kml,
  layer      = "rios_burgos_buffer",
  driver     = "KML",
  delete_dsn = TRUE
)
```

```
#> Deleting source `rios_burgos_buffer.kml' using driver `KML'
#> Writing layer `rios_burgos_buffer' to data source
#>   `rios_burgos_buffer.kml' using driver `KML'
#> Writing 4 features with 3 fields and geometry type Unknown (any).
```

```
message(">>> KML generado: ", ruta_kml)
```

---

17. Compresión a KMZ

KMZ es simplemente un archivo KML comprimido en ZIP con extensión `.kmz`. `zip::zip()` con `mode = "cherry-pick"` incluye solo el archivo KML especificado (sin rutas absolutas dentro del ZIP), lo que garantiza compatibilidad con Google Earth Pro y la función de importación de QGIS.

```
ruta_kmz <- "rios_burgos_buffer.kmz"
```

```r
zip::zip(
  zipfile = ruta_kmz,
  files   = ruta_kml,
  mode    = "cherry-pick"
)

message(">>> ¡KMZ generado correctamente!: ", ruta_kmz)
message(">>> Abre en Google Earth Pro o importa en QGIS.")
```

---

18. Resumen del flujo de trabajo

```
Shapefile IGN/CNIG


[1] st_read()             Lectura (opcionalmente filtrada por bbox)


[2] filter(grepl())       Selección de Arlanzón, Vena y Ubierna


[3] st_transform(25830)   Proyección métrica (ETRS89 UTM 30N)


[4] st_cast(MULTILINE)    Homogeneización de geometrías


[5] st_buffer(60 m)       Zona de ribera estimada


[6] st_union() / group    Disolución por río


[7] st_transform(4326)    Reproyección a WGS84


[8] mutate(OGR_STYLE)     Estilos de color para KML


[9] st_write() + zip()    Exportación KML → KMZ
```

---

19. Información de sesión

```
#> R version 4.5.2 (2025-10-31)
#> Platform: x86_64-pc-linux-gnu
#> Running under: Ubuntu 22.04.5 LTS
#>
#> Matrix products: default
#> BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.10.0
#> LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.10.0  LAPACK version 3.10.0
#>
```

```
#> locale:
#>  [1] LC_CTYPE=en_US.UTF-8       LC_NUMERIC=C
#>  [3] LC_TIME=en_GB.UTF-8        LC_COLLATE=en_US.UTF-8
#>  [5] LC_MONETARY=en_GB.UTF-8    LC_MESSAGES=en_US.UTF-8
#>  [7] LC_PAPER=en_GB.UTF-8       LC_NAME=C
#>  [9] LC_ADDRESS=C               LC_TELEPHONE=C
#> [11] LC_MEASUREMENT=en_GB.UTF-8 LC_IDENTIFICATION=C
#>
#> time zone: Europe/Madrid
#> tzcode source: system (glibc)
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods   base
#>
#> other attached packages:
#>  [1] zip_2.3.3       lubridate_1.9.4 forcats_1.0.1   stringr_1.6.0
#>  [5] dplyr_1.2.0     purrr_1.2.1     readr_2.1.6     tidyr_1.3.2
#>  [9] tibble_3.3.1    ggplot2_4.0.1   tidyverse_2.0.0 sf_1.1-0
#>
#> loaded via a namespace (and not attached):
#>  [1] utf8_1.2.6         generics_0.1.4     class_7.3-23       KernSmooth_2.23-26
#>  [5] stringi_1.8.7      hms_1.1.4          digest_0.6.39      magrittr_2.0.4
#>  [9] evaluate_1.0.5     grid_4.5.2         timechange_0.3.0   RColorBrewer_1.1-3
#> [13] fastmap_1.2.0      jsonlite_2.0.0     e1071_1.7-17       DBI_1.2.3
#> [17] tinytex_0.58       scales_1.4.0       cli_3.6.5          rlang_1.1.7
#> [21] units_1.0-0        withr_3.0.2        yaml_2.3.12        otel_0.2.0
#> [25] tools_4.5.2        tzdb_0.5.0         vctrs_0.7.1        R6_2.6.1
#> [29] proxy_0.4-29       lifecycle_1.0.5    classInt_0.4-11    pkgconfig_2.0.3
#> [33] pillar_1.11.1      gtable_0.3.6       glue_1.8.0         Rcpp_1.1.1
#> [37] xfun_0.56          tidyselect_1.2.1   knitr_1.51         farver_2.1.2
#> [41] htmltools_0.5.9    rmarkdown_2.30     compiler_4.5.2     S7_0.2.1
```